Overview

This Flask web application provides a user-friendly interface to control an LED strip connected to a Raspberry Pi. Users can visualize audio input from either:

1. Live audio captured by a USB microphone.

2. Uploaded `.wav` files.

The application also allows users to stop the visualization and ensures seamless operation across all features.

Features

1. Live Audio Visualization:

   - Captures real-time audio from a USB microphone.

   - Visualizes audio frequencies on the LED strip.

2. File Audio Visualization:

   - Allows users to upload `.wav` files.

   - Visualizes the audio content on the LED strip.

3. Stop Visualization:

   - Stops any ongoing visualization and resets the LED strip.

4. Responsive Web Interface:

   - Accessible via any device on the local network.

   - Clean, mobile-friendly design using a CSS stylesheet.

Code Components

1. Flask Application (app.py)

The `app.py` file is the main application. It defines the routes, visualization logic, and interactions with the hardware.

Routes

Route	Method	Description
`/`	GET	Displays the main menu for mode selection (live or file).
`/live`	POST	Starts the live audio visualization using the USB microphone.
`/file`	POST	Accepts a `.wav` file upload and starts visualization of the selected file.
`/stop`	POST	Stops any ongoing visualization and resets the LED strip.

Functions

1. ‘color_wheel(pos)`:
   - Generates a color gradient for the LED strip.

2. ‘process_audio_chunk(data, strip)`:
   - Processes a chunk of audio data and updates the LED strip based on its frequency spectrum.

3. `visualize_live_audio()`:
   - Captures live audio from the USB microphone and visualizes it.

4. `visualize_audio_file(file_path)`:
   - Reads a `.wav` file and visualizes its audio content.

5. `stop_visualization`:
   - Stops the currently running visualization thread safely.

2. HTML Templates

The templates are located in the `templates/` directory. Each template corresponds to a specific route.

Template	Description
`index.html`	Main menu page with options to start live visualization, upload a file, or stop visualization.
`live.html`	Displays the status of live audio visualization.
`file.html`	Displays the status of file-based audio visualization.
`404.html`	(Optional) Custom error page for invalid routes.
`settings.html`	(Optional) Allows users to adjust visualization settings like brightness and patterns.

3. Static Files

The static files are stored in the `static/` directory. The `styles.css` file provides consistent styling for the web interface.

Setup and Installation

1. Prerequisites

- Raspberry Pi (3B+ tested).

- WS2812 (WS2812B) or compatible LED strip.

- SunFounder Mini USB Microphone (or any compatible microphone).

- Python 3.7 or newer installed on the Raspberry Pi.

2. Install Required Python Libraries

- Run the following commands to install dependencies:

$ pip install flask numpy pyaudio rpi-ws281x

3. Hardware Setup

- Connect the LED strip to the Raspberry Pi and power supply via level shifter schematic:

- GPIO pin (GPIO 18 PWM0) for data.

- 5V and GND from a suitable power source.

- Connect the USB microphone to the Raspberry Pi.
- Ensure all components share a common ground.

How to Run

1. Start the Application:
   - Run the Flask application using:

$ python3 flask_app.py

2. Access the Web Interface:
   - Open a browser on any device connected to the same network.
   - Navigate to `http://<Raspberry_Pi_IP>:5000`.

3. Select a Mode:
   - Click "Start Live Audio Visualization" to visualize live microphone input.
   - Upload a `.wav` file for file-based visualization.

4. Stop Visualization:
   - Click "Stop Visualization" to halt any active visualization.

Usage Instructions

Main Page (`/`)

- Provides options to:
  - Start live audio visualization.
  - Upload a `.wav` file for playback.
  - Stop ongoing visualizations.

Live Audio Visualization (`/live`)

- Captures and visualizes live audio input from the USB microphone.

File Audio Visualization (`/file`)

- Allows uploading a `.wav` file for visualization.
- The file is stored in the `uploaded_files/` directory.

Stop Visualization (`/stop`)

- Stops any ongoing visualization and turns off the LEDs.

Customization

Change LED Strip Configuration

Modify the `LED_COUNT`, `LED_PIN`, `LED_BRIGHTNESS`, and other constants in `flask_app.py` to match your hardware setup.

Add New Patterns

Implement new patterns in the `process_audio_chunk` function or add new methods for advanced visualizations.

Customize Web Design

Update the `styles.css` file or add additional HTML templates for enhanced design and functionality.

Troubleshooting

Problem	Possible Cause	Solution
LEDs not lighting up	Incorrect GPIO pin or no power to LEDs	Verify connections and GPIO pin configuration.
No live audio visualization	USB microphone not detected	Check microphone connection using `arecord`.
`.wav` file not playing	Unsupported file format	Ensure the file is in `.wav` format.
Web app not accessible	Incorrect IP address or Flask not running	Check Raspberry Pi’s IP and restart Flask.